AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Algorytm listy dwukierunkowej z zastosowaniem GitHub

Autor:
Piotr Majchrzak

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **4**

1.1. Przykład 4

1.2. Instalacja projektu 4

2. Analiza problemu **5**

2.1. Zastosowanie listy dwukierunkowej 5

2.2. Sposób działania algorytmu 5

2.3. Przykład działania listy dwukierunkowej 5

3. Projektowanie **6**

3.1. Wykorzystywane narzędzia i technologie 6

3.2. Struktura projektu 6

3.3. Zastosowane wzorce projektowe 7

3.3.1. Wzorzec Factory 7

3.3.2. Wzorzec Iterator 7

3.4. Zarządzanie pamięcią 7

3.5. Konfiguracja kompilatora 8

3.6. Biblioteki 8

3.7. Diagram zależności klas 9

3.8. Schemat działania programu 9

3.9. Kontrola wersji za pomocą Git i GitHub 10

3.10. Generowanie dokumentacji za pomocą Doxygen 10

4. Implementacja **12**

4.1. Implementacja klasy Node 12

4.2. Inicjalizacja i zwalnianie pamięci 13

4.3. Dodawanie elementów do listy 13

4.4. Wyświetlanie elementów listy 16

4.5. Usuwanie elementów 17

4.6. Obsługa iteratora 19

4.7. Czyszczenie listy 20

4.8. Podsumowanie implementacji 20

5. Wnioski **22**

Literatura **23**

Spis rysunków **23**

Spis tabel **24**

Spis listingów **25**

1. Ogólne określenie wymagań

Program powinien implementować listę dwukierunkową w języku C++, działającą na stercie, z wykorzystaniem wzorców projektowych Factory i Iterator. Powinien umożliwiać dodawanie, usuwanie, wyświetlanie i czyszczenie elementów listy oraz poruszanie się po niej w obu kierunkach. Każda klasa powinna znajdować się w osobnym pliku, a jej działanie należy przetestować w funkcji main. Projekt powinien zawierać dokumentację wygenerowaną za pomocą Doxygen, dokumentację projektową oraz historię pracy z wykorzystaniem GitHuba.

1.1. Przykład

Lista dwukierunkowa może być wykorzystana do przechowywania i przeglądania liczb, z możliwością przechodzenia do następnego i poprzedniego elementu. Użytkownik może dodawać i usuwać liczby z wybranego miejsca oraz wyświetlać je w kolejności od początku do końca lub odwrotnie.

1.2. Instalacja projektu

Aby pobrać projekt z serwisu GitHub, należy uruchomić terminal i wpisać następujące polecenie:

1. Sklonuj repozytorium:

```
git clone https://github.com/p-majchrzak/PZ_projekt_Listy
```

2. Analiza problemu

2.1. Zastosowanie listy dwukierunkowej

Lista dwukierunkowa jest strukturą danych wykorzystywaną w programowaniu do przechowywania i organizowania elementów, które można przeglądać w obu kierunkach. Stosuje się ją między innymi w odtwarzaczach muzycznych, historii przeglądania stron internetowych, nawigacji między otwartymi dokumentami oraz w implementacji niektórych kolejek i list. Jej zaletą jest możliwość dynamicznego dodawania i usuwania elementów bez konieczności przesuwania pozostałych danych.

2.2. Sposób działania algorytmu

Lista dwukierunkowa składa się z węzłów, z których każdy przechowuje wartość oraz dwa wskaźniki: na poprzedni i następny element. Pierwszy węzeł nie posiada poprzednika, a ostatni nie posiada następnika. Dodawanie elementu polega na utworzeniu nowego węzła i odpowiednim ustawieniu wskaźników, natomiast usuwanie wymaga połączenia ze sobą sąsiednich węzłów. Przeglądanie listy odbywa się przez przechodzenie między węzłami, a czyszczenie polega na usunięciu wszystkich elementów i zwolnieniu zajmowanej pamięci.

2.3. Przykład działania listy dwukierunkowej

Przykładowo, lista może przechowować liczby 10, 20 i 30.

Początkowo lista jest pusta:

```
NULL
```

Po dodaniu liczb 10, 20 i 30 na koniec listy otrzymujemy:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

Przechodząc od początku, program wyświetli liczby w kolejności: 10, 20, 30. Przechodząc od końca, wyświetli: 30, 20, 10.

Jeżeli usuniemy element o wartości 20, wskaźniki zostaną zaktualizowane, a lista będzie wyglądać następująco:

```
NULL <- 10 <-> 30 -> NULL
```

W ten sposób można dodawać, usuwać i przeglądać liczby bez konieczności przesuwania pozostałych elementów.

3. Projektowanie

3.1. Wykorzystywane narzędzia i technologie

Projekt został opracowany w języku C++, który umożliwia tworzenie struktur danych oraz zarządzanie pamięcią dynamiczną. Program realizuje działanie listy dwukierunkowej przechowującej elementy w pamięci sterty.

Podczas realizacji projektu wykorzystywane są następujące narzędzia:

- **C++** – język programowania wykorzystany do implementacji listy dwukierunkowej oraz wzorców projektowych.
- **Kompilator C++** – narzędzie służące do przekształcania kodu źródłowego w program wykonywalny.
- **Git** – system kontroli wersji umożliwiający śledzenie zmian w kodzie, tworzenie commitów oraz przywracanie wcześniejszych wersji projektu.
- **GitHub** – platforma umożliwiająca przechowywanie repozytorium zdalnego, synchronizowanie zmian oraz pobieranie projektu w różnych lokalizacjach.
- **Doxygen** – narzędzie automatycznie generujące dokumentację na podstawie komentarzy i struktury kodu źródłowego.
- **LaTeX** – system składu tekstu wykorzystywany do przygotowania dokumentacji projektowej oraz wygenerowania dokumentacji w formacie PDF.

3.2. Struktura projektu

Projekt został podzielony na osobne pliki, aby oddzielić odpowiedzialności poszczególnych klas. Ułatwia to rozwijanie programu, testowanie jego działania oraz utrzymanie kodu.

W projekcie wykorzystywane są następujące klasy i pliki:

- **Node** – reprezentuje pojedynczy węzeł listy. Przechowuje wartość elementu oraz odwołania do poprzedniego i następnego węzła.
- **NodeFactory** – odpowiada za tworzenie nowych węzłów. Zastosowanie wzorca Factory oddziela proces tworzenia obiektów od pozostałej logiki programu.
- **Iterator** – umożliwia przechodzenie po elementach listy oraz obsługę operacji związanych z odczytem kolejnych i poprzednich elementów.

- **List** – implementuje listę dwukierunkową. Przechowuje wskaźniki na pierwszy i ostatni element oraz informacje o liczbie elementów. Udostępnia metody dodawania, usuwania, wyświetlania i czyszczenia listy.

- **main** – zawiera funkcję główną programu, w której tworzony jest obiekt listy i testowane są dostępne operacje.

3.3. Zastosowane wzorce projektowe

3.3.1. Wzorzec Factory

Wzorzec Factory został zastosowany do tworzenia węzłów listy. Zamiast umieszczać całą logikę tworzenia obiektów bezpośrednio w klasie `List`, program korzysta z klasy `NodeFactory` i jej metody `create()`.

Takie rozwiązanie oddziela tworzenie węzłów od operacji wykonywanych na liście. Ułatwia również modyfikowanie sposobu tworzenia obiektów w przyszłości.

3.3.2. Wzorzec Iterator

Wzorzec Iterator służy do poruszania się po elementach listy. Umożliwia odczytywanie kolejnych elementów oraz przechodzenie do elementu poprzedniego bez konieczności umieszczania całej logiki iteracji w klasie głównej programu.

W połączeniu z listą dwukierunkową pozwala to na obsługę przechodzenia w obu kierunkach.

3.4. Zarządzanie pamięcią

Elementy listy są tworzone dynamicznie w pamięci sterty. Każdy węzeł przechowuje wartość oraz dwa odwołania: do poprzedniego i następnego elementu.

Podczas dodawania elementu należy odpowiednio zaktualizować powiązania między węzłami. Przy usuwaniu elementu konieczne jest zachowanie poprawności połączeń pozostałych węzłów oraz zwolnienie pamięci zajmowanej przez usuwany obiekt.

Szczególną uwagę należy zwrócić na przypadki, w których lista jest pusta lub zawiera tylko jeden element. Metoda `clear()` powinna usuwać wszystkie elementy listy i przywracać jej stan początkowy.

3.5. Konfiguracja kompilatora

Kod źródłowy jest kompilowany za pomocą kompilatora zgodnego z językiem C++. Podczas kompilacji należy uwzględnić wszystkie pliki źródłowe zawierające implementacje klas oraz plik z funkcją `main()`.

Przykładowe polecenie kompilacji z wykorzystaniem kompilatora `g++` przedstawiono poniżej:

```
g++ -std=c++17 -Wall -Wextra -pedantic \
    main.cpp List.cpp Node.cpp NodeFactory.cpp Iterator.cpp \
    -o PZ_Projekt_Listy
```

Parametry kompilacji oznaczają:

- `-std=c++17` – wykorzystanie standardu C++17,
- `-Wall` – włączenie wielu ostrzeżeń kompilatora,
- `-Wextra` – włączenie dodatkowych ostrzeżeń,
- `-pedantic` – zgłaszanie problemów związanych z niezgodnością ze standardem języka,
- `-o PZ_Projekt_Listy` – określenie nazwy pliku wynikowego.

Podane polecenie przedstawia przykładową konfigurację kompilacji. Nazwy plików oraz standard języka należy dopasować do faktycznej struktury projektu i używanego środowiska.

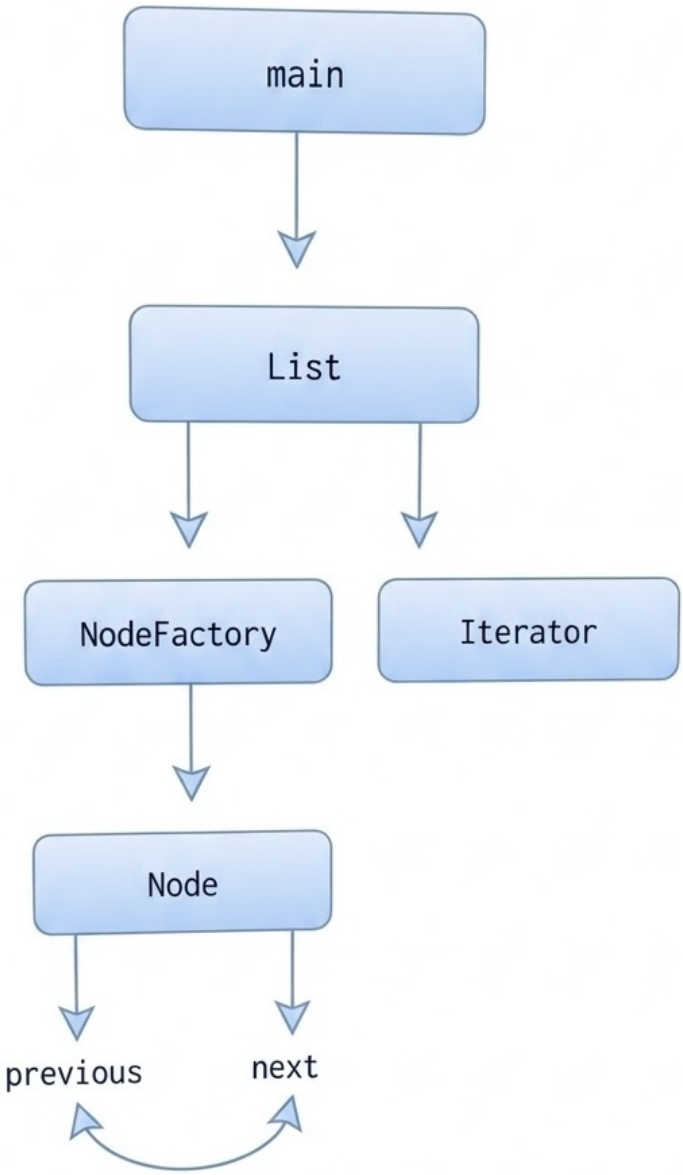
3.6. Biblioteki

Projekt może korzystać ze standardowych bibliotek języka C++, w zależności od zastosowanych instrukcji i sposobu wyświetlania wyników. Nie jest wymagane wykorzystywanie zewnętrznych bibliotek do samej implementacji listy dwukierunkowej.

W przypadku korzystania z biblioteki standardowej jej nagłówki należy dołączać w plikach, w których są potrzebne. Przykładowo biblioteka `<iostream>` umożliwia obsługę wejścia i wyjścia, natomiast `<cstddef>` udostępnia między innymi typ `std::size_t`.

3.7. Diagram zależności klas

Poniższy schemat przedstawia najważniejsze klasy projektu i ich zależności.



Rys. 3.1. Diagram zależności klas projektu

Klasa `List` odpowiada za organizację elementów i udostępnianie operacji na liście. `NodeFactory` uczestniczy w tworzeniu węzłów, a `Iterator` umożliwia poruszanie się po elementach. Węzły klasy `Node` są połączone w obu kierunkach.

3.8. Schemat działania programu

Działanie programu można przedstawić w następujących krokach:

1. Uruchomienie programu w funkcji `main()`.
2. Utworzenie obiektu listy dwukierunkowej.
3. Wykonanie wybranych operacji na liście.
4. W razie dodawania elementu utworzenie nowego węzła i połączenie go z odpowiednimi elementami.
5. W razie usuwania elementu aktualizacja połączeń oraz zwolnienie pamięci.
6. Wyświetlenie listy w kolejności od początku do końca lub od końca do początku.
7. Sprawdzenie działania iteratora przez przechodzenie do następnych i poprzednich elementów.
8. Usunięcie wszystkich elementów i zakończenie programu.

3.9. Kontrola wersji za pomocą Git i GitHub

Podczas realizacji projektu wykorzystywany jest system Git, który pozwala zapisywać kolejne etapy pracy nad programem. Każdy commit reprezentuje zapis zmian wprowadzonych w kodzie źródłowym.

Repozytorium GitHub umożliwia przechowywanie kopii projektu na serwerze oraz synchronizowanie zmian pomiędzy różnymi lokalizacjami. Dzięki temu można kontynuować pracę na pobranej kopii projektu, przesyłać zmiany do repozytorium zdalnego i pobierać aktualizacje do pozostałych kopii.

W ramach projektu Git służy również do sprawdzania działania operacji związanych z historią zmian, w tym przywracania wcześniejszych wersji oraz usuwania wybranych commitów. Pozwala to przeanalizować wpływ zmian w kodzie na poprawność kompilacji programu.

3.10. Generowanie dokumentacji za pomocą Doxygen

Doxygen jest wykorzystywany do automatycznego generowania dokumentacji na podstawie kodu źródłowego oraz komentarzy. Plik konfiguracyjny `Doxyfile` określa między innymi formaty dokumentacji i katalog wyjściowy.

W projekcie włączono generowanie dokumentacji HTML oraz LaTeX. Wygenerowane pliki LaTeX mogą następnie zostać skompilowane do dokumentu PDF.

Podstawowy proces generowania dokumentacji obejmuje:

1. Przygotowanie i uzupełnienie pliku `Doxyfile`.
2. Uruchomienie programu Doxygen.
3. Wygenerowanie plików dokumentacji.
4. Przejście do katalogu zawierającego pliki LaTeX.
5. Skompilowanie dokumentacji do formatu PDF.

Automatyczne generowanie dokumentacji ułatwia przedstawienie struktury projektu, klas, metod i zależności pomiędzy poszczególnymi elementami programu.

4. Implementacja

Implementacja projektu `PZ_Projekt_Listy` została wykonana w języku C++. Program wykorzystuje listę dwukierunkową, której elementy są przechowywane w pamięci dynamicznej. Każdy element zawiera wartość liczbową oraz wskaźniki umożliwiające przechodzenie do następnego i poprzedniego elementu.

Kod projektu został podzielony na pliki nagłówkowe `.hpp` oraz pliki źródłowe `.cpp`. W przedstawionej implementacji wykorzystano klasy `Node` i `List`, a także odwołania do klas `NodeFactory` oraz `Iterator`.

4.1. Implementacja klasy Node

Klasa `Node` reprezentuje pojedynczy element listy. Przechowuje wartość liczbową w polu `data` oraz dwa wskaźniki: `next` i `previous`. Pierwszy wskazuje następny element listy, a drugi element poprzedni.

Deklaracja klasy znajduje się w pliku `Node.hpp`.

```
#ifndef Node_hpp
#define Node_hpp

#include <stdio.h>

class Node{
public:
    int data;
    Node *next;
    Node *previous;

    Node(int number);
};

#endif /* Node_hpp */
```

Listing 1. Deklaracja klasy Node

Konstruktor klasy `Node` przyjmuje wartość liczbową, którą zapisuje w polu `data`. Oba wskaźniki są inicjalizowane wartością `nullptr`, ponieważ nowo utworzony element nie jest jeszcze połączony z innymi elementami listy.

```
#include "Node.hpp"

Node::Node(int number){
    data = number;
    next = nullptr;
```

```
6       previous = nullptr;
7   }
```

Listing 2. Konstruktor klasy Node

Takie rozwiązanie pozwala na późniejsze połączenie elementu z pozostałymi węzłami listy poprzez ustawienie odpowiednich wskaźników.

4.2. Inicjalizacja i zwalnianie pamięci

Konstruktor klasy `List` ustawia wskaźniki `first` oraz `last` na `nullptr`. Oznacza to, że lista początkowo nie zawiera żadnych elementów. Zmienna `size` otrzymuje wartość zero.

```
1   List::List(){
2       first = nullptr;
3       last = nullptr;
4       size = 0;
5   }
6
7   List::~List(){
8       clear();
9   }
```

Listing 3. Konstruktor i destruktor klasy List

Destruktor wywołuje metodę `clear()`, która usuwa wszystkie elementy listy i zwalnia zajmowaną przez nie pamięć. Dzięki temu po zakończeniu działania obiektu wykonywane jest czyszczenie dynamicznie zaalokowanych węzłów.

Do sprawdzania, czy lista jest pusta, służy metoda `isEmpty()`.

```
1   bool List::isEmpty(){
2       return first == nullptr;
3   }
```

Listing 4. Sprawdzanie, czy lista jest pusta

Metoda zwraca wartość `true`, gdy wskaźnik `first` nie wskazuje na żaden element. W przeciwnym przypadku zwracana jest wartość `false`.

4.3. Dodawanie elementów do listy

Metoda `addFirst()` dodaje nowy element na początku listy. W tym celu tworzy węzeł za pomocą metody `NodeFactory::create()`. Jeżeli lista jest pusta, nowy ele-

ment staje się jednocześnie pierwszym i ostatnim elementem. W przeciwnym przypadku zostaje połączony z dotychczasowym początkiem listy.

```
void List::addFirst(int value){
    Node *node = NodeFactory::create(value);

    if (isEmpty()){
        first = last = node;
    }else{
        node -> next = first;
        first->previous = node;
        first = node;
    }

    size++;
    iterator.setCurrent(first);
}
```

Listing 5. Dodawanie elementu na początku listy

Po dodaniu elementu zwiększana jest wartość `size`. Następnie aktualizowana jest pozycja iteratora za pomocą metody `setCurrent()`.

Dodawanie elementu na końcu listy realizuje metoda `addLast()`. Nowy węzeł zostaje połączony z dotychczasowym ostatnim elementem. Wskaźnik `last` zostaje następnie ustawiony na nowy węzeł.

```
void List::addLast(int value){
    Node *node = NodeFactory::create(value);

    if(isEmpty()){
        first = last = node;
    }else{
        node->previous = last;
        last->next = node;
        last = node;
    }

    size++;
    iterator.setCurrent(first);
}
```

Listing 6. Dodawanie elementu na końcu listy

Obie metody obsługują przypadek pustej listy, dzięki czemu pierwszy dodawany element prawidłowo ustawia oba wskaźniki graniczne.

Metoda `addAt()` umożliwia wstawienie elementu pod wskazanym indeksem. Naj-

pierw sprawdzana jest poprawność indeksu. Wartości mniejsze od zera oraz większe od aktualnego rozmiaru listy są odrzucane. Dla indeksu zerowego wywoływana jest metoda `addFirst()`, natomiast dla indeksu równego rozmiarowi listy — metoda `addLast()`.

```
void List::addAt(int index, int value){
    if(index < 0 || index > size){
        cout << "Nieprawidlowy indeks!" << endl;
        return;
    }

    if(index == 0){
        addFirst(value);
        return;
    }

    if(index == size){
        addLast(value);
        return;
    }
```

Listing 7. Sprawdzanie indeksu w metodzie addAt

W przypadku wstawiania elementu wewnątrz listy program przechodzi do węzła znajdującego się pod wskazanym indeksem. Następnie tworzy nowy element i aktualizuje połączenia między sąsiednimi węzłami.

```
    Node *current = first;

    for(int i = 0; i < index; i++){
        current = current->next;
    }

    Node *node = NodeFactory::create(value);

    node->next = current;
    node->previous = current->previous;
    current->previous->next = node;
    current->previous = node;

    size++;
    iterator.setCurrent(first);
}
```

Listing 8. Wstawianie elementu wewnątrz listy

Dzięki aktualizacji obu kierunków połączeń lista zachowuje swoją dwukierunkową strukturę.

4.4. Wyświetlanie elementów listy

Do wyświetlania zawartości listy od początku do końca służy metoda `showAll()`. Przechodzi ona przez kolejne węzły, zaczynając od `first`, aż do momentu napotkania wartości `nullptr`. Każdy element jest wyświetlany wraz z numerem porządkowym.

```
void List::showAll(){
    int counter = 1;
    Node *current = first;

    while(current != nullptr){
        cout << counter << ") "
             << current->data << endl;

        current = current->next;
        counter++;
    }
}
```

Listing 9. Wyświetlanie listy w przód

Wyświetlanie w odwrotnym kierunku realizuje metoda `showReverse()`. W tym przypadku przechodzenie rozpoczyna się od wskaźnika `last`, a kolejne elementy są odczytywane za pomocą wskaźnika `previous`.

```
void List::showReverse(){
    int counter = 1;
    Node *current = last;

    while(current != nullptr){
        cout << counter << ") "
             << current->data << endl;

        current = current->previous;
        counter++;
    }
}
```

Listing 10. Wyświetlanie listy w odwrotnej kolejności

Wykorzystanie obu wskaźników pozwala na przechodzenie przez tę samą listę w dwóch kierunkach bez konieczności tworzenia jej dodatkowej kopii.

4.5. Usuwanie elementów

Metoda `removeFirst()` usuwa pierwszy element listy. Na początku sprawdza, czy lista nie jest pusta. Jeśli nie zawiera elementów, wyświetlany jest odpowiedni komunikat i metoda kończy działanie.

```
void List::removeFirst(){
    if(isEmpty()){
        cout << "Lista jest pusta!" << endl;
        return;
    }

    Node *temp = first;

    if(first == last){
        first = nullptr;
        last = nullptr;
    }else{
        first = first->next;
        first->previous = nullptr;
    }

    delete temp;
    size--;
    iterator.setCurrent(first);
}
```

Listing 11. Usuwanie pierwszego elementu

W przypadku listy zawierającej tylko jeden element oba wskaźniki zostają ustawione na `nullptr`. W przeciwnym przypadku początek listy zostaje przesunięty na kolejny węzeł, a jego wskaźnik `previous` zostaje wyzerowany. Usunięty węzeł jest zwalniany operatorem `delete`.

Metoda `removeLast()` wykonuje analogiczną operację na końcu listy.

```
void List::removeLast(){
    if(isEmpty()){
        cout << "Lista jest pusta!" << endl;
        return;
    }

    Node *temp = last;

    if(first == last){
        first = nullptr;
        last = nullptr;
```

```
    }else{
        last = last->previous;
        last->next = nullptr;
    }

    delete temp;
    size--;
    iterator.setCurrent(first);
}
```

Listing 12. Usuwanie ostatniego elementu

W przypadku listy wieloelementowej wskaźnik `last` zostaje przesunięty na poprzedni węzeł. Następnie jego wskaźnik `next` jest ustawiany na `nullptr`.

Usuwanie elementu pod określonym indeksem realizuje metoda `removeAt()`. Najpierw sprawdzana jest poprawność indeksu. Jeżeli wskazany element znajduje się na początku lub na końcu listy, wykorzystywane są odpowiednio metody `removeFirst()` i `removeLast()`.

```
void List::removeAt(int index){
    if(index < 0 || index >= size){
        cout << "Nieprawidlowy indeks!" << endl;
        return;
    }

    if(index == 0){
        removeFirst();
        return;
    }

    if(index == size - 1){
        removeLast();
        return;
    }
```

Listing 13. Sprawdzanie indeksu w metodzie removeAt

Dla elementu znajdującego się wewnątrz listy program odnajduje odpowiedni węzeł i łączy ze sobą jego poprzednika oraz następnika. Następnie usuwa wskazany element i aktualizuje rozmiar listy.

```
    Node *current = first;

    for(int i = 0; i < index; i++){
        current = current->next;
    }
```

```
6
7       current->previous->next = current->next;
8       current->next->previous = current->previous;
9
10      delete current;
11      size--;
12      iterator.setCurrent(first);
13  }
```

Listing 14. Usuwanie elementu ze środka listy

Dzięki odpowiedniemu przepięciu wskaźników usunięcie węzła nie powoduje przerwania połączeń pomiędzy pozostałymi elementami listy.

4.6. Obsługa iteratora

W klasie `List` wykorzystano obiekt `iterator`, który umożliwia przechodzenie do kolejnych i poprzednich elementów. Metody `showNext()` oraz `showPrevious()` sprawdzają, czy możliwe jest wykonanie danego kroku.

```
1  void List::showNext(){
2      if(iterator.hasNext()){
3          cout << "Nastepny element: "
4               << iterator.next() << endl;
5      }else{
6          cout << "Brak nastepnego elementu!"
7               << endl;
8      }
9  }
```

Listing 15. Przejście do następnego elementu

Jeżeli iterator może przejść do następnego elementu, metoda `next()` zwraca jego wartość. W przeciwnym przypadku wyświetlany jest komunikat o braku następnego elementu.

Analogicznie działa przechodzenie w przeciwnym kierunku.

```
1  void List::showPrevious(){
2      if(iterator.hasPrevious()){
3          cout << "Poprzedni element: "
4               << iterator.previous() << endl;
5      }else{
6          cout << "Brak poprzedniego elementu!"
7               << endl;
8      }
```

```
}
```

Listing 16. Przejście do poprzedniego elementu

Po wykonaniu operacji dodawania lub usuwania elementów pozycja iteratora jest ustawiana ponownie za pomocą `iterator.setCurrent(first)`. Pełna implementacja działania iteratora znajduje się w osobnych plikach projektu.

4.7. Czyszczenie listy

Metoda `clear()` usuwa wszystkie elementy listy. Przechodzi przez kolejne węzły, zapamiętuje adres aktualnego elementu, przesuwa wskaźnik na następny węzeł, a następnie usuwa zapamiętany element.

```
void List::clear(){
    Node *current = first;

    while(current != nullptr){
        Node *temp = current;
        current = current->next;
        delete temp;
    }

    first = nullptr;
    last = nullptr;
    size = 0;
    iterator.setCurrent(nullptr);
}
```

Listing 17. Usuwanie wszystkich elementów listy

Po zakończeniu pętli wskaźniki `first` i `last` są ustawiane na `nullptr`, a rozmiar listy zostaje wyzerowany. Iterator również otrzymuje wartość `nullptr`, co odpowiada pustej liście.

4.8. Podsumowanie implementacji

Przedstawiona implementacja umożliwia dodawanie, usuwanie i wyświetlanie elementów listy dwukierunkowej. Wykorzystanie wskaźników `next` i `previous` umożliwia przechodzenie w obu kierunkach, natomiast wskaźniki `first` i `last` zapewniają bezpośredni dostęp do początku i końca struktury.

Tworzenie nowych węzłów odbywa się przez `NodeFactory::create()`, a obsługa przechodzenia pomiędzy elementami została powiązana z obiektem `iterator`. Pa-

mięć zajmowana przez elementy jest zwalniana podczas ich usuwania oraz przy czyszczeniu całej listy.

5. Wnioski

Celem projektu było zaimplementowanie listy dwukierunkowej w języku C++ z wykorzystaniem wzorców projektowych Factory oraz Iterator. W ramach projektu utworzono klasy odpowiedzialne za przechowywanie elementów listy, zarządzanie jej zawartością oraz poruszanie się pomiędzy kolejnymi elementami.

Zaimplementowane operacje umożliwiają dodawanie i usuwanie elementów na początku, na końcu oraz pod wskazanym indeksem. Program pozwala również na wyświetlanie zawartości listy w obu kierunkach, przechodzenie do następnego i poprzedniego elementu oraz usuwanie wszystkich elementów.

Podczas realizacji projektu wykorzystano wskaźniki i dynamiczne zarządzanie pamięcią. Istotnym elementem implementacji było prawidłowe aktualizowanie połączeń pomiędzy węzłami, aby zachować strukturę listy po wykonaniu operacji dodawania i usuwania. Zastosowanie destruktora oraz metody `clear()` umożliwia zwalnianie pamięci zajmowanej przez elementy listy.

Projekt pozwolił na praktyczne zastosowanie programowania obiektowego, podziału kodu na pliki nagłówkowe i źródłowe oraz wykorzystanie wzorców projektowych. W rezultacie powstała struktura danych obsługująca podstawowe operacje na liście dwukierunkowej, stanowiąca przykład zastosowania języka C++ do tworzenia własnych struktur danych.

Tab. 5.1. Podsumowanie funkcjonalności projektu

Funkcjonalność	**Opis**
Dodawanie elementów	Początek, koniec i wybrany indeks
Usuwanie elementów	Początek, koniec i wybrany indeks
Wyświetlanie listy	Kolejność od początku do końca
Wyświetlanie odwrotne	Kolejność od końca do początku
Iterator	Przejście do następnego i poprzedniego elementu
Czyszczenie listy	Usunięcie wszystkich elementów
NodeFactory	Tworzenie nowych węzłów listy
Zarządzanie pamięcią	Zwalnianie pamięci usuwanych węzłów

Spis rysunków

3.1. Diagram zależności klas projektu 9

Spis tabel

5.1. Podsumowanie funkcjonalności projektu 22

Spis listingów

1. Deklaracja klasy Node 12
2. Konstruktor klasy Node 12
3. Konstruktor i destruktor klasy List 13
4. Sprawdzanie, czy lista jest pusta 13
5. Dodawanie elementu na początku listy 14
6. Dodawanie elementu na końcu listy 14
7. Sprawdzanie indeksu w metodzie addAt 15
8. Wstawianie elementu wewnątrz listy 15
9. Wyświetlanie listy w przód 16
10. Wyświetlanie listy w odwrotnej kolejności 16
11. Usuwanie pierwszego elementu 17
12. Usuwanie ostatniego elementu 17
13. Sprawdzanie indeksu w metodzie removeAt 18
14. Usuwanie elementu ze środka listy 18
15. Przejście do następnego elementu 19
16. Przejście do poprzedniego elementu 19
17. Usuwanie wszystkich elementów listy 20